

UNIT – I : MACHINE LEARNING BASICS

(What is machine learning? Key terminology, Key tasks of machine learning, how to choose right algorithm, steps in developing a machine learning why python? Getting started with NumPy library Classifying with k- Nearest Neighbors: The k-Nearest Neighbors classification algorithm, Parsing and importing data from a text file, creating scatter plots with Matplotlib, Normalizing numeric values)

1. Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to **learn automatically from data and improve performance with experience, without being explicitly programmed**.

Traditional programming requires rules and logic to be manually coded, whereas Machine Learning allows a system to **discover patterns and relationships** in data and use them for decision-making or prediction. The basic idea is: *“Learn from past data (experience) and apply that knowledge to new, unseen data.”*

A computer program is said to learn from **experience (E)** with respect to some **task (T)** and a **performance measure (P)** if its performance on T, as measured by P, improves with experience E.

Example:

- **Task (T):** Predicting whether an email is spam.
 - **Experience (E):** Past labelled emails (spam / not spam).
 - **Performance measure (P):** Accuracy of predictions.
-

Key Features of Machine Learning

1. **Data-Driven** – Learns patterns from raw data.
 2. **Adaptive** – Improves performance with more data.
 3. **Automation** – Reduces need for explicit programming.
 4. **Predictive Power** – Can predict unseen cases.
 5. **Wide Applications** – Healthcare, Finance, Robotics, NLP, etc.
-

Types of Machine Learning

1. **Supervised Learning** – Learning from labelled data (e.g., email spam detection).
 2. **Unsupervised Learning** – Finding hidden patterns in unlabelled data (e.g., clustering customers).
 3. **Reinforcement Learning** – Learning by interacting with an environment and receiving feedback/rewards (e.g., game-playing AI).
 4. **Semi-Supervised Learning** – Mix of labelled and unlabelled data.
-

Examples

- Predicting house prices from features (size, location).
 - Identifying faces in images.
 - Speech recognition (voice assistants).
 - Product recommendations (shopping or streaming platforms).
-

Advantages

- Learns complex patterns automatically.
 - Improves accuracy with more data.
 - Reduces manual effort of programming rules.
 - Can adapt to new scenarios.
-

Applications of Machine Learning

- **Healthcare**: Disease prediction, medical image analysis.
 - **Finance**: Fraud detection, stock market prediction.
 - **E-commerce**: Recommendation systems.
 - **Transportation**: Self-driving vehicles.
 - **Natural Language Processing (NLP)**: Chatbots, translation.
-

2. Key Terminology in Machine Learning

Machine Learning involves a set of key terms that form the foundation for understanding and applying algorithms. These terminologies help in clearly defining the problem, the data, and the evaluation of results.

- **Algorithm** – A sequence of steps or rules followed by a computer to perform a learning task.
- **Model** – The mathematical representation created by the learning algorithm based on data. It is used to make predictions.
- **Training Data** – The dataset used to teach the model. It contains both inputs (features) and outputs (labels).
- **Testing Data** – A separate dataset used to evaluate how well the model performs on unseen data.
- **Features (Attributes/Independent Variables)** – The input variables used by the model to make predictions.
- **Labels (Target/Dependent Variable)** – The output variable the model is trying to predict.
- **Overfitting** – When a model learns the training data too well, including noise, and performs poorly on new data.
- **Underfitting** – When a model is too simple and fails to capture the underlying trend in data.
- **Supervised Learning** – Learning with labelled data, where both input and output are provided.
- **Unsupervised Learning** – Learning with unlabelled data, where the algorithm finds hidden patterns or structure.
- **Classification** – Predicting categorical outcomes (e.g., spam or not spam).
- **Regression** – Predicting continuous outcomes (e.g., house prices).
- **Clustering** – Grouping data points based on similarity (e.g., customer segmentation).
- **Evaluation Metrics** – Measures used to assess model performance, such as accuracy, precision, recall, and F1-score.

- Understanding these terms is essential for correctly applying ML algorithms.

- Helps in designing experiments, interpreting results, and communicating effectively in the field of data science.

3. Key Tasks of Machine Learning

Machine Learning can be applied to solve different types of problems. These are classified into key tasks based on the nature of data and output required.

- **Classification**
 - The task of assigning input data into predefined categories.

Example: Identifying whether an email is spam or not.
- **Regression**
 - Predicting a continuous numeric value based on input features.

Example: Estimating house prices based on size and location.
- **Clustering**
 - Grouping similar data points together without predefined labels.

Example: Segmenting customers based on purchasing behavior.
- **Dimensionality Reduction**
 - Reducing the number of input variables while preserving important information.

Example: Compressing images or reducing features in a dataset.
- **Anomaly Detection**
 - Identifying rare or unusual patterns in data.

Example: Detecting fraudulent transactions in banking.
- **Recommendation**
 - Suggesting items or actions based on user preferences and data.

Example: Movie recommendations on streaming platforms.
- **Reinforcement Learning Tasks**
 - Learning through trial and error by interacting with an environment and receiving feedback in the form of rewards or penalties.

Example: Training robots to walk or play games.

The key tasks of machine learning define the scope of problems that can be solved using algorithms. By identifying the right task, we can apply the most suitable method for achieving accurate and useful results.

4. How to Choose the Right Algorithm

Choosing the correct machine learning algorithm is a critical step that directly impacts the performance and accuracy of the model. The choice depends on several factors such as the type of data, problem requirements, and computational resources.

Factors to Consider

- **Type of Problem**
 - **Classification** → Decision Trees, Naïve Bayes, Logistic Regression, SVM.
 - **Regression** → Linear Regression, Decision Trees, Random Forests.
 - **Clustering** → K-Means, Hierarchical Clustering.
 - **Recommendation** → Collaborative Filtering, Matrix Factorization.
- **Size and Nature of Data**
 - Large datasets → Neural Networks or ensemble methods.
 - Small datasets → Simpler models like Decision Trees or Logistic Regression.
 - High-dimensional data → Algorithms that handle dimensionality reduction (PCA, SVM with kernels).
- **Accuracy vs. Interpretability**
 - If accuracy is more important → Complex models (Neural Networks, Ensembles).
 - If interpretability is important → Simpler models (Decision Trees, Linear Regression).
- **Training Time and Resources**
 - Fast training needed → Naïve Bayes, Logistic Regression.
 - If more resources available → Deep Learning, SVM with kernels.
- **Handling Missing Data or Noise**
 - Some algorithms like Decision Trees can naturally handle missing values.
 - Others require preprocessing and imputation.
- **Scalability**
 - Algorithms like KNN may not scale well with very large datasets.
 - Neural Networks and Gradient Boosted Trees scale better.

General Guidelines

1. Start with a simple model.
2. Evaluate performance using cross-validation.
3. If results are not satisfactory, try more complex algorithms.
4. Balance between accuracy, interpretability, and computational cost.

NOTE

There is no single best algorithm for all problems. The choice depends on the specific nature of the task, dataset, and project goals. Often, multiple algorithms are tested to determine which works best.

5. Steps in Developing a Machine Learning Model

Steps to Build a Machine Learning Model - GeeksforGeeks – [click on this link to get more info](#)

Developing or creating a machine learning model involves a systematic process to ensure accuracy, efficiency, and reliability. The process usually follows the below steps:

1. Define the Problem

- Clearly state the objective.
- Identify whether it is a classification, regression, clustering, or recommendation task.
- Understand business or research goals.

2. Collect Data

- Gather relevant and sufficient data from available sources (databases, files, sensors, web, etc.).
- Ensure data quality because poor data leads to poor results.

3. Data Preprocessing

- Clean data: Handle missing values, remove duplicates, and manage noise.
- Normalize or standardize values for consistency.
- Encode categorical values into numerical form.
- Perform feature selection or dimensionality reduction.

4. Split the Data

- Divide data into training and testing sets.
- Sometimes also keep a validation set.
- Typical split: 70% training, 30% testing (varies by case).

5. Select the Algorithm

- Choose based on type of problem, size of data, and performance requirements.
- Start with simple models and then move to complex ones if needed.

6. Train the Model

- Feed the training dataset into the algorithm.
- The model learns patterns, relationships, and parameters.

7. Evaluate the Model

- Test on unseen data (testing set).
- Use metrics like accuracy, precision, recall, F1-score, RMSE, or AUC depending on the problem.

8. Optimize the Model

- Tune hyperparameters for better performance.
- Use techniques like Grid Search, Random Search, or Bayesian Optimization.
- Apply regularization to prevent overfitting.

9. Deploy the Model

- Integrate the model into applications or decision-making systems.
- Ensure it works with real-world data.

10. Monitor and Maintain

- Continuously monitor model performance.
- Update or retrain the model when new data becomes available.
- Handle concept drift (when data patterns change over time).

Developing a machine learning model is an iterative process. A successful model requires proper problem understanding, quality data, suitable algorithm selection, and continuous improvement after deployment.

6. Why Python for Machine Learning?

Python has become the most popular programming language for machine learning due to its simplicity, flexibility, and wide ecosystem of tools. It is widely adopted by researchers, developers, and industries.

Python is the preferred language for machine learning because it is simple, powerful, and supported by a rich ecosystem. Its flexibility and strong community support make it ideal for beginners as well as experts.

1. Ease of Learning and Simplicity

- Python has a simple and readable syntax, making it beginner-friendly.
- The focus remains on solving machine learning problems rather than struggling with complex code.

2. Rich Libraries and Frameworks

- Python provides numerous libraries that support data analysis, visualization, and machine learning.
- Popular libraries include NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, and PyTorch.
- These libraries reduce the need to code everything from scratch.

3. Cross-Platform and Flexibility

- Python works on all major operating systems.
- It can integrate with other languages like C, C++, and Java.
- Supports both object-oriented and functional programming styles.

4. Community Support

- Python has a very large and active community.
- A wealth of tutorials, forums, and documentation is available for learning and problem-solving.

5. Strong Data Handling Capabilities

- Python can easily handle large datasets.
- Libraries like Pandas and NumPy allow efficient manipulation of data.
- Tools like Matplotlib and Seaborn help visualize data for better understanding.

6. Integration with Machine Learning and AI

- Python integrates well with advanced ML and AI frameworks.
- Supports deep learning, reinforcement learning, and natural language processing.
- Provides APIs for easy deployment in applications.

7. Industry and Research Adoption

- Widely used in companies, research labs, and academia.
 - Supported by cloud platforms such as AWS, Azure, and Google Cloud.
 - Availability of pre-trained models and tools speeds up project development.
-

7. Getting Started with NumPy Library

NumPy (Numerical Python) is one of the most important Python libraries for scientific computing and machine learning. It provides support for arrays, mathematical functions, and tools to efficiently work with large datasets.

1. What is NumPy?

- A Python library designed for numerical computations.
- It introduces **ndarray**, a powerful n-dimensional array object.
- Much faster and more memory-efficient compared to Python lists.

2. Key Features of NumPy

- Provides fast operations on large arrays and matrices.
- Includes mathematical, logical, and statistical functions.
- Supports broadcasting (performing operations on arrays of different shapes).
- Can interface with C/C++ and Fortran code for high performance.

3. Why NumPy is Important in Machine Learning

- Machine learning requires large amounts of data processing, and NumPy makes this efficient.
- Many ML libraries (like Scikit-learn, TensorFlow, PyTorch) are built on top of NumPy.
- Useful for data preprocessing, mathematical modeling, and handling datasets.

4. Creating Arrays in NumPy

```
import numpy as np

# Creating arrays
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])

# Special arrays
zeros = np.zeros((2, 2))
ones = np.ones((3, 3))
random = np.random.rand(2, 3)
```

5. Basic Operations in NumPy

```
# Array operations
arr = np.array([1, 2, 3, 4])
print(arr + 5)      # Add scalar
print(arr * 2)      # Multiply scalar
print(arr ** 2)     # Power operation

# Mathematical functions
print(np.mean(arr)) # Average
print(np.std(arr))  # Standard deviation
print(np.sum(arr))  # Sum
```

6. Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])      # First element
print(arr[1:4])    # Slice elements
```

7. NumPy Applications in ML

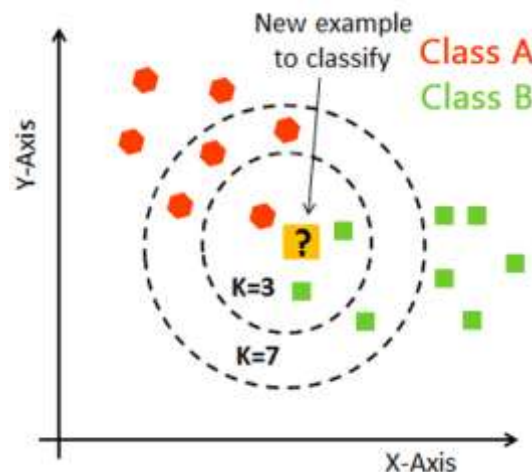
- Data preprocessing (normalization, scaling).
- Matrix operations in linear algebra.
- Feature transformation and manipulation.
- Working with large numerical datasets efficiently.

NumPy is the foundation of numerical computing in Python. Its array operations, speed, and integration with other ML libraries make it a crucial tool for machine learning.

8. k-Nearest Neighbors (k-NN) Classification Algorithm

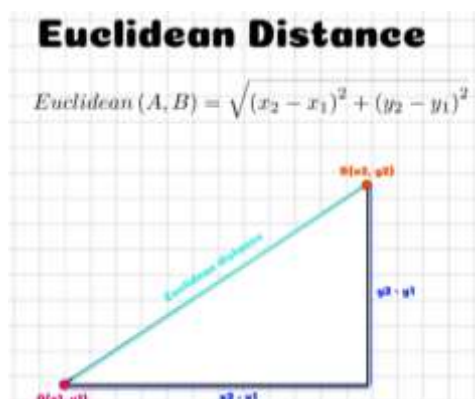
The **k-Nearest Neighbors (k-NN)** algorithm is a simple, non-parametric, and instance-based learning technique used for classification and regression. It is based on the idea that data points with similar features are likely to belong to the same class.

The **k-NN algorithm** is one of the most intuitive classification methods. While simple, it can be highly effective when combined with proper data preprocessing and distance metrics.

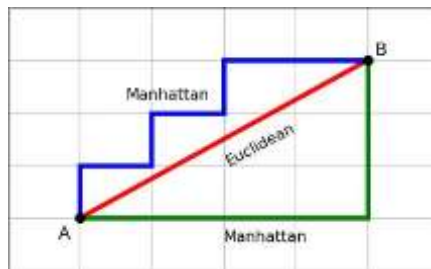


Working Principle

- The algorithm does not build an explicit model.
- It stores the entire training dataset and makes predictions based on the closest data points to the query instance.
- “Closeness” is measured using a **distance metric** such as:
 - **Euclidean Distance**



- **Manhattan Distance**



$$M_{\text{dist}} = |x_2 - x_1| + |y_2 - y_1|$$

- **Minkowski Distance**

$$\text{Minkowski Distance} = \left(\sum_{i=1}^n |x_i - y_i| \right)^{1/p}$$

Steps in the Algorithm

1. Choose the number **k** (the number of neighbors to consider).
2. Calculate the distance between the query instance and all training data points.
3. Identify the **k nearest neighbors** based on the smallest distances.
4. Count the frequency of each class among these neighbors.
5. Assign the class that has the majority vote.

Example (Conceptual)

Suppose we want to classify a new fruit based on its size and colour.

- Training data: Apples (small, red), Oranges (medium, orange), Mangoes (large, yellow).
- Query: medium-sized, orange-coloured fruit.
- If **k = 3**, and two nearest neighbours are oranges, one is apple → Prediction = Orange.

Choosing the Value of k

- Small k → sensitive to noise, risk of overfitting.
- Large k → smoother decision boundary but may ignore local patterns.

Usually chosen using cross-validation.

Advantages

- Simple to understand and implement.
- No training phase (lazy learning).
- Works well with multi-class problems.

Limitations

- Computationally expensive for large datasets (since distances must be calculated for all points).
- Requires careful selection of k.
- Sensitive to irrelevant features and feature scaling.

Applications

- Image and text classification.
- Medical diagnosis.

9. Parsing and Importing Data from a Text File

In machine learning, data is often stored in external files such as text files, CSV files, or JSON files. Before analysis, the data must be **parsed (interpreted)** and **imported (loaded)** into the program so that it can be used for training and testing.

Parsing and importing data from a text file is a crucial preprocessing step in machine learning. It ensures that the raw data is transformed into a structured form suitable for further analysis and model training.

Parsing Data

- **Parsing** means breaking the data into meaningful parts that can be processed by the algorithm.
- For example, in a text file where each line contains a record, parsing will separate values by spaces, commas, or tabs.

Importing Data

- **Importing** is the process of reading the file and storing it into suitable data structures (like lists, arrays, or data frames).
- This allows the data to be manipulated, cleaned, and used for training machine learning models.

Steps in Parsing and Importing a Text File

1. **Open the file** – Load the file into the program using built-in file handling functions.
2. **Read the contents** – Read the file line by line.
3. **Split data** – Separate each line into features using a delimiter (comma, tab, space).
4. **Convert to numerical values** – If necessary, convert strings into integers or floats.
5. **Store in data structures** – Save the parsed data into arrays, lists, or data frames.
6. **Use for ML tasks** – Feed the data into algorithms for classification, regression, or other tasks.

Example (Conceptual)

Suppose a text file contains the following student data:

```
Name, Marks, Grade
Sirisha, 85, A
Tejasri, 72, B
Aishwarya, 90, A
Hima bindu, 65, C
```

- Parsing will separate each line into fields:
(Sirisha, 85, A), (Tejasri, 72, B), (Aishwarya, 90, A), (Hima bindu, 65, C).
- Importing will convert them into a structured format, e.g., a table or matrix.

Importance in Machine Learning

- Data is the foundation of ML; without proper parsing, algorithms cannot understand raw text.
- Ensures data is in a **clean, structured, and usable** format.

Advantages

- Allows flexible handling of different file formats.
- Enables automation of reading large datasets.
- Makes preprocessing and analysis easier.

Challenges

- Handling missing or corrupted data.
- Converting categorical text into numerical form.

10. Creating Scatter Plots with Matplotlib

Visualization is an important part of machine learning. Scatter plots are one of the simplest and most effective ways to visualize relationships between two variables. By plotting points on a two-dimensional graph, patterns, trends, or groupings in data can be observed.

What is a Scatter Plot?

- A scatter plot displays individual data points as dots in a two-dimensional plane.
- The **x-axis** represents one feature, while the **y-axis** represents another.
- Scatter plots are especially useful for identifying clusters, outliers, or correlations.

Steps in Creating a Scatter Plot

1. Import the plotting library (Matplotlib).
 2. Prepare the dataset with two numerical variables.
 3. Call the `scatter()` function to plot points.
 4. Label the axes and give a title.
 5. Display the graph.
-

Example Dataset

Consider marks obtained by four students in **Math** and **Science** subjects:

Name	Math	Science
Rama Lakshmi	82	90
Chandana	75	70
Jahnavi	88	85
Sadvika	60	65

- X-axis → Math scores
- Y-axis → Science scores

Example Code (Python)

```
import matplotlib.pyplot as plt

# Data
names = ["Rama Lakshmi", "Chandana", "Jahnavi", "Sadvika"]
math_scores = [82, 75, 88, 60]
science_scores = [90, 70, 85, 65]

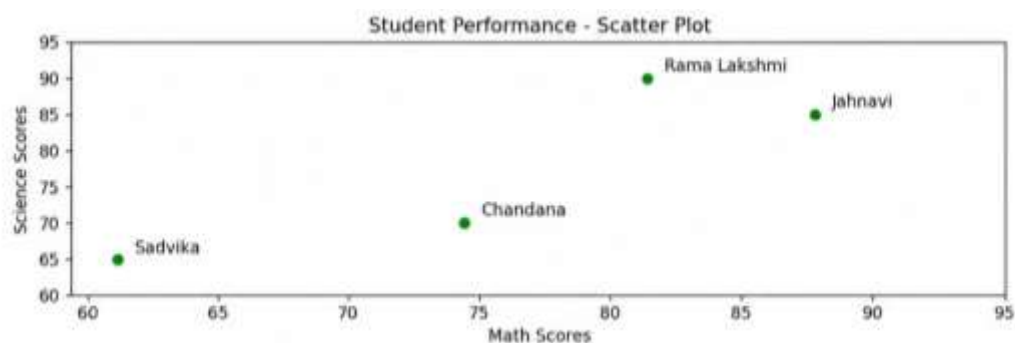
# Scatter plot
plt.scatter(math_scores, science_scores, color='green')

# Adding Labels for each student
for i, name in enumerate(names):
    plt.text(math_scores[i] + 0.5, science_scores[i] + 0.5, name)

# Titles and Labels
plt.xlabel("Math Scores")
plt.ylabel("Science Scores")
plt.title("Student Performance - Scatter Plot")

# Display
plt.show()
```

Output



Here are the output graphs for topic 10:

- The **first graph** shows the original student marks.
- The **second graph** shows the marks sorted with bin ranges applied (red dashed lines for bins).
- The scatter plot will show four points, each representing a student.
- Rama Lakshmi will appear at (82, 90), Chandana at (75, 70), Jahnavi at (88, 85), and Sadvika at (60, 65).
- From the graph, we can see that **Rama Lakshmi is strong in science**, while **Jahnavi is balanced in both subjects**.

Importance of scatter plots in Machine Learning

- Helps understand **data distribution** before applying algorithms.
- Useful in visualizing **classification boundaries**.
- Can detect outliers or errors in data.

Advantages

- Simple to create and interpret.
- Reveals relationships between variables.
- Effective for small datasets.

Limitations

- Not suitable for very large datasets (points may overlap).
- Can only show relationships between two variables at a time.

Scatter plots are a powerful tool for visually understanding data. They provide insights into trends, patterns, and possible groupings, which guide further machine learning tasks like classification and clustering.

11. Normalizing Numeric Values

In machine learning, different features often have different scales. For example, one feature may represent **age in years (20–60)**, while another may represent **salary in thousands (10,000–100,000)**. Since most algorithms use distance or similarity measures, having features in different scales can cause **bias**. To avoid this, we normalize numeric values.

What is Normalization?

Normalization is the process of adjusting values measured on different scales to a common scale, usually between **0 and 1**.

- It ensures that no single feature dominates the learning process.
- Helps algorithms like **k-Nearest Neighbors, Logistic Regression, and SVM** perform better.

Formula for Min-Max Normalization

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

Where:

- x = original value
- x' = normalized value (between 0 and 1)
- $\min(X)$ = minimum value in the dataset
- $\max(X)$ = maximum value in the dataset

Example Dataset

Suppose we have **four students** with their **study hours per week**:

Name	Study Hours
Mounika	10
Mohitha	25
Navya	30
Sharmila	15

- Minimum value = 10 (Mounika)
- Maximum value = 30 (Navya)

Normalization Step

For each student:

- Mounika $\rightarrow (10-10) / (30-10) = 0$
- Mohitha $\rightarrow (25-10) / (30-10) = 15/20 = 0.75$
- Navya $\rightarrow (30-10) / (30-10) = 1$
- Sharmila $\rightarrow (15-10) / (30-10) = 5/20 = 0.25$

Normalized Dataset

Name	Study Hours	Normalized Value
Mounika	10	0.00
Mohitha	25	0.75
Navya	30	1.00
Sharmila	15	0.25

Python Example

```
import numpy as np

# Data
names = ["Mounika", "Mohitha", "Navya", "Sharmila"]
study_hours = np.array([10, 25, 30, 15])

# Normalization using min-max scaling
min_val = study_hours.min()
max_val = study_hours.max()
normalized = (study_hours - min_val) / (max_val - min_val)

# Display results
for i in range(len(names)):
    print(f"{names[i]}: Original = {study_hours[i]}, Normalized = {normalized[i]:.2f}")
```

Output (Conceptual)

- Mounika → 0.00
- Mohitha → 0.75
- Navya → 1.00
- Sharmila → 0.25

Mounika: Original = 10, Normalized = 0.00

Mohitha: Original = 25, Normalized = 0.75

Navya: Original = 30, Normalized = 1.00

Sharmila: Original = 15, Normalized = 0.25

Importance in Machine Learning

- Prevents one feature from dominating due to larger scale.
- Improves accuracy of distance-based algorithms.
- Makes gradient descent optimization converge faster.

Normalization is a crucial preprocessing step in machine learning. By scaling values between 0 and 1, it ensures fair comparison across features and enhances the performance of models.